

Replit prompt:

We need to build a **SharePoint document-management layer** for OpenClaw using the existing **assistant@ Microsoft Graph identity**.

This is **not** about email sending or calendar invites.

This is specifically about letting OpenClaw **read, create, update, and append to account/opportunity files stored in SharePoint**, while preserving auditability and version history.

The goal is to make **SharePoint the source of truth** for rich account and opportunity notes, while the local CRM remains only a **summary metadata layer**.

OVERALL GOAL

Build a reliable Graph-backed file access layer so OpenClaw can:

1. **List** files/folders in a SharePoint location
2. **Read** an existing account/opportunity file
3. **Create** a new file if it doesn't exist
4. **Update** an existing file using content from a temp file
5. **Append** new notes / meeting summaries to an existing file using content from a temp file

All actions should run as **assistant@** so the audit trail and version history show the assistant as the actor.

IMPORTANT DESIGN REQUIREMENTS

1. SharePoint is the source of truth

The detailed notes for:

- opportunities
- accounts
- clients

should live in SharePoint, **not primarily in local markdown files**.

The local CRM should only hold:

- company
- contact
- stage
- last touch
- next step
- campaign eligibility
- SharePoint path

2. No destructive operations

This system must **NOT** allow:

- delete
- rename
- move
- silent overwrite without version history

Allowed operations:

- list
- read
- create
- update
- append

If SharePoint versioning is enabled, updates should naturally create new versions.

3. Use file-first pattern

Do **not** pass long content directly in shell arguments.

Use temp files like:

-

`/tmp/oc-sp-content.txt`

The OpenClaw workflow should be:

1. write content to temp file with the

`write`
`tool`

2. run a short clean command via `exec` that points to the temp file

This is the same design pattern we now use for longer emails and calendar invites.

4. Assistant identity only

All SharePoint operations should use the existing **assistant@ Microsoft Graph auth** path.

Do not build this around Tom's identity.

The point is:

- audit trail
- clean ownership
- assistant activity is attributable
- version history reflects assistant@ activity

FOLDER / FILE MODEL

Assume SharePoint will use a structure like:

```
/Stackstone CRM/  
  Opportunities/  
    Harken Health.md  
    Butcombe Group.md  
    Avanti Recruitment.md  
  Accounts/  
    Croyde Medical.md  
    Andy Barrett - SJP.md
```

Start with **one file per opportunity/account**.

Do not overcomplicate version 1 with multiple files per account.

Each file can contain structured markdown sections such as:

- Summary
- Contacts
- Timeline
- Notes
- Risks / objections
- Next steps
- Commercial

IMPLEMENTATION REQUIRED

Build the file access capability under:

```
~/openclaw/integrations/microsoft-l1/
```

You can either build:

Option A — one unified script

Example:

```
python3 ~/openclaw/integrations/microsoft-l1/sharepoint.py list
"<sharepoint-path>"
python3 ~/openclaw/integrations/microsoft-l1/sharepoint.py read
"<sharepoint-file-path>"
python3 ~/openclaw/integrations/microsoft-l1/sharepoint.py create
"<sharepoint-file-path>" --content-file /tmp/oc-sp-content.txt
python3 ~/openclaw/integrations/microsoft-l1/sharepoint.py update
"<sharepoint-file-path>" --content-file /tmp/oc-sp-content.txt
python3 ~/openclaw/integrations/microsoft-l1/sharepoint.py append
"<sharepoint-file-path>" --content-file /tmp/oc-sp-content.txt
```

Option B — separate scripts

- sharepoint-list.py
- sharepoint-read.py
- sharepoint-create.py

- sharepoint-update.py

- sharepoint-append.py

I slightly prefer **Option A** because it is easier to document and use in skills.

FUNCTIONAL REQUIREMENTS

A. LIST

Command should:

- list folders/files in a given SharePoint path
- return readable names/paths
- help OpenClaw discover whether a file already exists

Example target:

```
python3 ../sharepoint.py list "/Stackstone CRM/Opportunities"
```

B. READ

Command should:

- fetch the full content of a given SharePoint markdown file
- print it safely to stdout (or save locally if needed)
- support paths like:

```
•  
/Stackstone CRM/Opportunities/Harken Health.md
```

Example:

```
python3 ../sharepoint.py read "/Stackstone CRM/Opportunities/Harken  
Health.md"
```

C. CREATE

Command should:

- create a new markdown file from a temp content file
- fail safely if the file already exists unless an overwrite flag is explicitly provided
- be suitable for initialising new opportunities/accounts

Example:

```
python3 ../sharepoint.py create "/Stackstone CRM/Opportunities/Harken
Health.md" --content-file /tmp/oc-sp-content.txt
```

D. UPDATE

Command should:

- replace the full file content with the content of a temp file
- rely on SharePoint version history for safety
- return a success message with path + version info if available

Example:

```
python3 ../sharepoint.py update "/Stackstone CRM/Opportunities/Harken
Health.md" --content-file /tmp/oc-sp-content.txt
```

E. APPEND

Command should:

- read the existing file
- append new content from the temp file cleanly
- preserve readable formatting
- write it back as a new version

This is important for:

- meeting notes
- call summaries
- incremental updates

Example:

```
python3 ../sharepoint.py append "/Stackstone CRM/Opportunities/Harken  
Health.md" --content-file /tmp/oc-sp-content.txt
```

GRAPH / AUTH REQUIREMENTS

Use the existing Microsoft Graph auth that already powers:

- assistant@ email send
- assistant@ calendar invite creation

Do not invent a separate login system if the existing token/auth path can be reused.

Please identify what Graph permissions are needed.

Likely permissions:

- Files.ReadWrite.All
- Sites.ReadWrite.All

But use the minimum viable set if possible.

Tell me clearly:

1. what scopes are required
2. whether the current assistant@ auth already has them
3. whether re-auth is needed

SAFETY / BEHAVIOUR RULES

Build these constraints into the implementation:

Must allow:

- list
- read
- create
- update
- append

Must not allow:

- delete
- rename
- move

Logging / output

Each command should return:

- the action taken
- the resolved SharePoint path
- success/failure
- any version ID or metadata if available

Errors should be helpful

For example:

- file not found
- folder not found
- auth missing
- permission denied
- invalid path
- content file missing

OPENCLAW USAGE MODEL

This is how OpenClaw/L1 will use it later:

For a new opportunity

1. check if the SharePoint file exists
2. if not, create it
3. update CRM summary locally with:
 - stage
 - next step
 - last touch

- campaign eligibility
- SharePoint path

For updates after calls

1. read the current SharePoint file
2. append or update notes
3. keep the detailed context in SharePoint
4. keep local CRM summary lean

For paid work

If paid work is booked or delivered:

- move entity from opportunity logic to account logic
- create/update file under

`/Stackstone CRM/Accounts/...`

The SharePoint layer needs to support this cleanly.

DO NOT DO THESE THINGS

- Do not build a delete function
- Do not build GUI-only logic; we need CLI/script usage
- Do not require long raw content in shell args
- Do not use heredocs as the main execution path
- Do not create local shadow copies as the primary record
- Do not make the Pi file system the source of truth for these notes

WHAT I WANT BACK FROM YOU

Please implement the scripts and then tell me:

1. What files you created

List exact paths under:

~/openclaw/integrations/microsoft-11/

2. Exact commands to use

For:

- list
- read
- create
- update
- append

3. Graph permissions/scopes required

And whether the current assistant@ auth already supports them

4. Any re-auth steps needed

If OpenClaw / assistant@ needs new consent or onboarding

5. How to test it

Give me a step-by-step test using a sample file like:

- /Stackstone CRM/Opportunities/Test Opportunity.md

Include:

- create test file
- read it
- append to it
- update it
- verify version history behaviour

6. Any assumptions you made

About SharePoint site path, drive ID, root folder, auth tokens, etc.

SUCCESS CONDITION

This is successful when OpenClaw can do the following safely:

1. You tell it an account/opportunity needs a note file
2. It creates or reads the file in SharePoint as **assistant@**
3. It updates the file via temp-file-driven command
4. The change is versioned in SharePoint
5. The local CRM only stores the summary + SharePoint path

That is the target.

Make the solution robust and practical. This will become part of the production operating system for Stackstone CRM and account management.

If you want, after this I can also draft the **matching**

`crm-sharepoint`

skill that will sit on top of whatever Replit builds.